Memory Efficient Doubly Linked List –

Delete At End – Space Complexity Analysis and Algorithm

Program:

```
void deleteFromEnd()
{
    if (head == nullptr)
    {
        cout << "List is empty. Nothing to delete." << endl;
        return;
    }

    Node *curr = head;
    Node *prev = nullptr;
    Node *next;

    // Traverse to end
    while (true)
    {
        next = XOR(prev, curr->npx);
        if (next == nullptr)
            break; // curr is the last node
        prev = curr;
        curr = next;
    }

    // curr is the last node. prev is the second to last.
    if (prev != nullptr)
    {
        // prev->npx was: XOR(prevPrev, curr)
        // Needs to be: XOR(prevPrev, NULL)

        // Decode prevPrev
        Node *prevPrev = XOR(prev->npx, curr);
        prev->npx = XOR(prevPrev, nullptr);
    }
```

```
    else
    {
        // List had only one node
        head = nullptr;
    }

    cout << "Deleted " << curr->data << " from the end." <<
endl;
    free(curr);
}

….
        case 6:
            deleteFromEnd();
            break;
```

***Space Complexity Analysis*: *XOR Linked List* (*Delete From End*)**

1. *Input Space Complexity*

- ***Parameter – only Definition***:
  - ***Function Parameters***: *None (takes no arguments).*
  - ***Analysis***: *No additional data is passed into the function.*
  - ***Complexity***: $O(1)$

- ***Full Data Definition***:
  - ***Function Data***: *The existing XOR linked list structure.*
  - ***Analysis***: *The function must traverse all n nodes of the list to locate the last node.The entire list serves as the input context.*
  - ***Complexity***: $O(n)$

---

2. *Auxiliary Space Complexity*

Explanation: *This measures the extra memory used by the algorithm during execution, excluding the input itself.*

- ***Local Variables***: *The function utilizes three local pointers:* *curr*, *prev*, *and* *next*. *Additionally, it uses a temporary* *prevPrev* *pointer during the linkage update.*
    - ***Analysis***: *These are stack − allocated pointers. Their number is fixed (constant) regardless of whether the list contains* 10 *nodes or* 10,000 *nodes.*

- ***Traversal***: *The traversal is handled via a* *while*(*true*) *loop (iteration). Since it is not recursive, it does not add any frames to the call stack.*

- ***XOR Operations***: *Bitwise XOR calculations are performed within CPU registers and do not require heap or additional stack allocation.*

- ***Deallocation***: *free*(*curr*) *returns the memory of the last node to the heap. While this reduces the total memory footprint, the auxiliary space used to perform the logic is unchanged.*

Conclusion: *The amount of extra memory used is fixed and does not scale with n.*

- ***Auxiliary Space Complexity***: $\boldsymbol{O(1)}$

Total Space Complexity

Total Space Complexity = Auxiliary Space + Input Space

Based on the definition of input space you use, the total space complexity changes.

- **Common Interpretation (Parameter-only input):**

$$\underbrace{\boldsymbol{Space} = \boldsymbol{O(1)}}_{\textit{Auxiliary}} \quad + \quad \underbrace{\boldsymbol{O(1)}}_{\textit{Input}} = \boldsymbol{O(1)}.$$

- **Holistic Interpretation (Full-data input):**

$$\underbrace{\boldsymbol{Space} = \boldsymbol{O(1)}}_{\textit{Auxiliary}} \quad + \quad \underbrace{\boldsymbol{O(n)}}_{\textit{Input}} = \boldsymbol{O(n)}.$$

In most academic and interview contexts, when asked for the space complexity of a function, the **auxiliary space complexity (O(1))** is the expected answer.

***Algorithm*: *Delete From End* (*XOR Doubly Linked List*)**

***XORDELEND*(*START*):**

1. [***CHECK IF LIST IS EMPTY***]
   If START = NULL, *then*:
   Write: "*List is empty. Nothing to delete.*"
   Return and ***EXIT***.

   [*End of If structure*]

2. [***INITIALIZE TRAVERSAL***]
   Set CURRPTR := START
   Set PREVPTR := NULL

3. [***TRAVERSE TO THE LAST NODE***]

   Repeat while TRUE:
   a. Set NEXTTEMPPTR := PREVPTR ⊕ NEXTXORPTR[CURRPTR]
   b. If NEXTTEMPPTR = NULL, *then*:
   Break (*CURRPTR is now the last node*)
   c. Set PREVPTR := CURRPTR
   d. Set CURRPTR := NEXTTEMPPTR
   [*End of Loop*]

4. [***CHECK IF LIST HAD ONLY ONE NODE***]
   If PREVPTR = NULL, *then*:
   Set START := NULL (*The only node in the list is being deleted*)

5. [***UPDATE SECOND − TO − LAST NODE LINKAGE***]
   Else:

   a. Set PREVPREVTEMPPTR := NEXTXORPTR[PREVPTR] ⊕ CURRPTR (*Decode the node preceding the second to last node.*)

   b. Set NEXTXORPTR[PREVPTR] := PREVPREVTEMPPTR ⊕ NULL (*Update the second − to − last node to be the new last node by setting its `next` to NULL*)

   [*End of If structure*]

6. [***FREE MEMORY AND PRINT***]
   $Write$: "$Deleted$ ", $INFO[CURRPTR]$, " $from\ the\ end$."

   $$\boldsymbol{FREE}(CURRPTR)\begin{pmatrix} Return\ the\ memory \\ of\ the\ deleted\ node \\ to\ the\ system \end{pmatrix}$$

7. ***EXIT***
